

Blocco 16 - Dashboard e capstone NoSQL

Aggregazioni MongoDB per monitoraggio telemetrico

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Dal dato alla dashboard

Una dashboard NoSQL non nasce da magia grafica: nasce da aggregazioni controllate e ripetibili.

Obiettivo del blocco

Problema

Costruire query di dashboard sulla telemetria: KPI, ultima lettura, trend, vista energia, vista ambiente, alert e controlli qualità.

Idea guida

Ogni card deve avere una pipeline esplicita, una granularità dichiarata e un criterio di verifica.

Set minimo di dashboard

- ① KPI generale: letture, dispositivi, warning;
- ② tabella ultima lettura per dispositivo;
- ③ trend a bucket di 5 minuti;
- ④ card energia per sito;
- ⑤ card ambiente per sito;
- ⑥ alert recenti;
- ⑦ controllo raw-curved.

Granularità delle card

Card	Una riga rappresenta	Tipo
KPI generale	intero sistema	numero
Ultima lettura	dispositivo	tabella
Trend	bucket temporale	linea
Energia per sito	sito e tipo dispositivo	tabella/barre
Ambiente per sito	sito e regione	tabella/barre
Alert recenti	alert	tabella

2. Aggregation pipeline

La pipeline è il corrispettivo NoSQL della query dichiarativa per dashboard.

Card 1: KPI generale

```
db.readings_curated.aggregate([
  {
    $group: {
      _id: null,
      readings: { $sum: 1 },
      devices: { $addToSet: "$device_id" },
      warnings: {
        $sum: { $cond: [{ $eq: ["$status", "warning"] }, 1, 0] }
      }
    }
  },
  { $project: { _id: 0, readings: 1, device_count: { $size: "$devices" }, warnings: 1 } }
]);
```

Perché questa pipeline è corretta

- ▶ legge la collection curata, non il raw;
- ▶ conta tutte le letture disponibili;
- ▶ usa `$addToSet` per non contare due volte lo stesso dispositivo;
- ▶ conta i warning con una condizione esplicita;
- ▶ restituisce una sola riga, adatta a KPI.

Card 2: ultima lettura

```
db.readings_curated.aggregate([
  { $sort: { device_id: 1, ts: -1 } },
  {
    $group: {
      _id: "$device_id",
      device_type: { $first: "$device_type" },
      site: { $first: "$site" },
      region: { $first: "$region" },
      ts: { $first: "$ts" },
      status: { $first: "$status" },
      metrics: { $first: "$metrics" }
    }
  },
  { $sort: { _id: 1 } }
]);
```

Ragionamento sulla card ultima lettura

La pipeline ordina prima per dispositivo e timestamp decrescente. Poi raggruppa per dispositivo e usa `$first`.

Se dimentichiamo l'ordinamento, `$first` non significa “più recente”: significa solo “primo documento incontrato”. Questo è un errore frequente nelle pipeline.

Card 3: trend temporale

```
db.readings_curated.aggregate([
  {
    $group: {
      _id: {
        bucket: { $dateTrunc: { date: "$ts", unit: "minute", binSize: 5 } }
      },
      readings: { $sum: 1 },
      warnings: { $sum: { $cond: [{ $eq: ["$status", "warning"] }, 1, 0] } }
    }
  },
  { $project: { _id: 0, bucket: "$_id.bucket", readings: 1, warnings: 1 } },
  { $sort: { bucket: 1 } }
]);
```

- ▶ un punto per ogni evento è troppo rumoroso;
- ▶ un bucket raggruppa eventi in finestre temporali;
- ▶ la dimensione del bucket dipende dal caso: 1 minuto, 5 minuti, 1 ora;
- ▶ bucket troppo piccoli generano rumore;
- ▶ bucket troppo grandi nascondono anomalie.

3. Card per energia e ambiente

La stessa collection contiene metriche diverse: la dashboard deve rispettare il tipo dispositivo.

Card energia

```
db.readings_curated.aggregate([
  { $match: { device_type: { $in: ["transformer", "solar_inverter", "smart_meter"] } } },
  {
    $group: {
      _id: { site: "$site", device_type: "$device_type" },
      readings: { $sum: 1 },
      avg_temperature_c: { $avg: "$metrics.temperature_c" },
      avg_power_kw: { $avg: "$metrics.power_kw" },
      avg_load_kw: { $avg: "$metrics.load_kw" },
      avg_consumption_kw: { $avg: "$metrics.consumption_kw" }
    }
  },
  { $sort: { "_id.site": 1, "_id.device_type": 1 } }
]);
```

- ▶ non tutti i dispositivi hanno `power_kw`;
- ▶ il trasformatore ha `load_kw`;
- ▶ lo smart meter ha `consumption_kw`;
- ▶ il modello documentale accetta questa variabilità;
- ▶ la dashboard deve spiegare i valori nulli.

Card ambiente

```
db.readings_curated.aggregate([
  { $match: { device_type: { $in: ["air_quality_station", "weather_station"] } } },
  {
    $group: {
      _id: { site: "$site", region: "$region" },
      readings: { $sum: 1 },
      avg_temperature_c: { $avg: "$metrics.temperature_c" },
      avg_humidity_pct: { $avg: "$metrics.humidity_pct" },
      max_pm25_ugm3: { $max: "$metrics.pm25_ugm3" },
      max_wind_kmh: { $max: "$metrics.wind_kmh" }
    }
  },
  { $sort: { "_id.region": 1, "_id.site": 1 } }
]);
```


- ▶ PM2.5 è presente solo nelle centraline aria;
- ▶ vento è presente nelle stazioni meteo;
- ▶ temperatura e umidità possono essere comuni;
- ▶ il raggruppamento per sito evita medie territoriali troppo generiche;
- ▶ le metriche massime evidenziano picchi.

4. Controlli e qualità

La dashboard deve controllare la pipeline, non solo visualizzare numeri.

Raw-curated

```
{
  raw_events: db.readings_raw.countDocuments(),
  curated_readings: db.readings_curated.countDocuments(),
  raw_without_curated: db.readings_raw.aggregate([
    {
      $lookup: {
        from: "readings_curated",
        localField: "raw_id",
        foreignField: "raw_id",
        as: "curated"
      }
    },
    { $match: { curated: { $size: 0 } } },
    { $count: "missing" }
  ]).toArray()
}
```

Perché controllare raw-curated

- ▶ raw maggiore di curated può indicare ritardo di trasformazione;
- ▶ curated maggiore di raw può indicare duplicazione o caricamenti impropri;
- ▶ un raw senza curated può essere errore, scarto o dato non valido;
- ▶ il controllo va discusso insieme alla politica di qualità.

Alert recenti

```
db.alerts.find(  
  {},  
  {  
    _id: 0,  
    alert_id: 1,  
    device_id: 1,  
    ts: 1,  
    severity: 1,  
    rule: 1,  
    message: 1  
  }  
)  
.sort({ ts: -1 }).limit(10);
```

5. Capstone NoSQL

L'obiettivo finale è consegnare un sistema replicabile e spiegabile, non solo query isolate.

Consegna finale

- 1 avviare lo stack telemetry;
- 2 caricare o rigenerare il database;
- 3 lanciare il collector e verificare nuovi dati;
- 4 eseguire le aggregation pipeline;
- 5 costruire una dashboard concettuale con card e granularità;
- 6 spiegare indici, limiti e controlli qualità.

Cosa deve contenere la presentazione

- ▶ diagramma dei container;
- ▶ schema delle collection;
- ▶ esempio documento raw e curated;
- ▶ elenco card con pipeline associata;
- ▶ criterio di verifica di ogni card;
- ▶ motivazione della scelta NoSQL;
- ▶ confronto con la soluzione ticketing relazionale.

Errori frequenti

- ▶ usare raw direttamente come fonte della dashboard;
- ▶ aggregare metriche non confrontabili tra dispositivi diversi;
- ▶ usare `$first` senza ordinare;
- ▶ non dichiarare la granularità della card;
- ▶ dimenticare retention e qualità dati;
- ▶ proporre NoSQL senza spiegare quale query migliora.

- ▶ Le aggregation pipeline sostituiscono le query SQL nel caso MongoDB.
- ▶ Ogni card deve avere granularità e controlli.
- ▶ Il modello documentale gestisce metriche variabili ma richiede disciplina.
- ▶ Il capstone NoSQL replica il percorso ticketing con una tecnologia diversa.